Assignment No. 3

Name : Ashutosh Sanjay More **PRN :** 124B1F036

Batch: A2 **Div :** A **Course :** Design and Analysis of Algorithms

Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git

---

Emergency Relief Supply Distribution Using Fractional Knapsack

1. Problem Statement

A devastating flood has affected multiple villages in a remote area. The government, along with various NGOs, is organizing an emergency relief operation to provide essential supplies to the affected people.

A rescue team has a boat with a limited carrying capacity of **W kilograms**. The boat must transport critical relief supplies such as food, medicine, and drinking water from a relief center to the affected villages.

Each type of relief item has:

- **Weight (wi):** The weight of the item in kilograms.
- **Utility Value (vi):** The importance or usefulness of the item. For example, medicine may have a higher utility value than food or water.
- **Divisibility:** Some items, such as food and water, can be divided into smaller portions, while other items, such as medical kits, must be transported as complete units.

The objective is to select supplies in such a way that the total utility value is maximized without exceeding the boat's weight capacity.The Fractional Knapsack algorithm uses a greedy strategy by selecting items according to their utility value per kilogram.

---

2. Goals

The main goals of this assignment are:

1. To implement the Fractional Knapsack algorithm for emergency relief supply distribution.
2. To maximize the total utility value of supplies transported by the boat.
3. To prioritize items having a high utility value per kilogram.
4. To allow partial selection of divisible items such as food and water.
5. To ensure that the available boat capacity is utilized efficiently.
6. To demonstrate the application of the Greedy Method to a real-world emergency management problem.

---

3. Course Objectives

The objectives of this assignment are:

- To understand the basics of computational complexity of various algorithms.
- To understand the Greedy Method of algorithm design.
- To select an appropriate algorithm design strategy for solving real-world optimization problems.
- To analyze the efficiency and performance of an algorithm.
- To understand the Fractional Knapsack problem and its practical applications.

---

4. Course Outcomes

After completing this assignment, the student will be able to:

- Analyze the asymptotic performance of algorithms.
- Understand and apply the Greedy Method.
- Solve optimization problems using suitable algorithmic strategies.
- Apply the Fractional Knapsack algorithm to real-world problems.
- Determine the time and space complexity of an algorithm.
- Develop an efficient solution for resource allocation under capacity constraints.

---

5. Theory

Fractional Knapsack Problem

The Fractional Knapsack Problem is a classic optimization problem in which a set of items is given. Each item has a weight and a corresponding value. A knapsack or container has a maximum weight capacity.

The objective is to select items such that:

- The total weight does not exceed the capacity.
- The total value or utility is as large as possible.

In the **Fractional Knapsack Problem**, an item can be divided into smaller portions. Therefore, if an entire item cannot fit into the remaining capacity, a fraction of the item can be selected.

For example, if 10 kg of food has a utility value of 30 and only 5 kg of capacity remains, then half of the food can be transported and its utility value will be:

Fraction selected = 5 / 10 = 0.5

Utility obtained = 0.5 × 30 = 15

The Fractional Knapsack problem can be solved optimally using the **Greedy Method**.

Greedy Strategy

The greedy strategy calculates the **utility-to-weight ratio** of every item:

Utility-to-weight ratio = Utility Value / Weight

The item having the highest utility per kilogram is given the highest priority.

The steps are:

1. Calculate the utility-to-weight ratio for every item.
2. Sort all items in descending order of their ratio.
3. Select the item with the highest ratio.
4. If the complete item fits in the remaining capacity, take it completely.
5. If the item does not fit and it is divisible, take the fraction that fits.

6. If the item is indivisible and does not fit, skip it.
7. Continue until the boat is full or all items have been considered.

Greedy-Choice Property

The Fractional Knapsack problem satisfies the greedy-choice property because selecting the item with the highest utility per kilogram first cannot reduce the optimality of the solution.

For example, if one item provides 10 units of utility per kilogram and another provides 3 units per kilogram, selecting the first item gives a better utility for every kilogram of available capacity.

Optimal Substructure

The problem also has optimal substructure.

Once the highest utility-per-weight item has been selected, the remaining capacity forms a smaller Fractional Knapsack problem. The same greedy strategy can then be applied to the remaining items.

---

6. Mathematical Formulation

Let there be **n** items.

For each item:

- wi = weight of item i
- vi = utility value of item i
- `xi = fraction of item i selected, where 0 ≤ xi ≤ 1`
- W = maximum capacity of the boat

The objective is to maximize:

Maximize:

$Z = \Sigma (vi \times xi)$

Subject to:

$\Sigma (wi \times xi) \leq W$

and

$0 \leq xi \leq 1$

For indivisible items, xi can only be 0 or 1.

---

7. Algorithm

Algorithm: MaxUtilFractional

Input:
A list of items containing weight, utility value, and divisibility status, and the boat capacity W.

Output:
Maximum total utility value.

Steps:

1. Start.
2. Read the number of items and boat capacity W.
3. For every item, calculate:
   `ratio = utility / weight`
4. Sort all items in descending order of utility-to-weight ratio.
5. Set `totalUtility = 0`.
6. Set `remainingCapacity = W`.
7. Traverse the sorted list of items.
8. If the complete item fits:
   - Take the complete item.
   - Add its utility to `totalUtility`.

    - Reduce the remaining capacity.
9. Otherwise, if the item is divisible:
    - Calculate the fraction that can fit.
    - Add the corresponding utility.
    - Set the remaining capacity to zero.
10. Otherwise, skip the indivisible item.
11. Stop when the boat becomes full or all items have been processed.
12. Display the maximum utility.
13. Stop.

---

8. Pseudocode

```
function MaxUtilFractional(items, W):

    for each item in items:
        item.ratio = item.utility / item.weight

    sort items in descending order of item.ratio

    totalUtility = 0
    remainingCapacity = W

    for each item in items:

        if remainingCapacity == 0:
            break

        if item.weight <= remainingCapacity:

            totalUtility = totalUtility + item.utility
            remainingCapacity = remainingCapacity - item.weight

        else if item.isDivisible == true:

            fraction = remainingCapacity / item.weight

            totalUtility = totalUtility + (fraction * item.utility)
```

```
            remainingCapacity = 0
        else:
            continue
    return totalUtility
```

9. Sample Example

Consider the following relief supplies:

Item	Weight (kg)	Utility Value	Divisible
Medicine	5	50	No
Food	10	30	Yes
Water	20	20	Yes

The capacity of the boat is:

W = 17 kg

Step 1: Calculate Utility per Kilogram

For Medicine:

Utility/Weight = 50/5 = **10**

For Food:

Utility/Weight = 30/10 = **3**

For Water:

Utility/Weight = 20/20 = **1**

Therefore, the preference order is:

Medicine → Food → Water

Step 2: Fill the Boat

Select Medicine

Medicine weighs 5 kg and has a utility of 50.

`Since 5 kg ≤ 17 kg, the complete medicine kit is selected.`

Weight used = 5 kg

Utility obtained = 50

Remaining capacity:

17 - 5 = **12 kg**

Select Food

Food weighs 10 kg and has a utility of 30.

`Since 10 kg ≤ 12 kg, the complete food supply is selected.`

Weight used = 10 kg

Utility obtained = 30

Remaining capacity:

12 - 10 = **2 kg**

Select Water

Water weighs 20 kg, but only 2 kg of capacity remains.

Water is divisible, so only 2 kg of water is selected.

Fraction selected:

2/20 = **0.1**

Utility obtained:

0.1 × 20 = **2**

Remaining capacity:

2 - 2 = **0 kg**

The boat is now full.

---

10. Result

Item	Total Weight	Utility	Selection	Utility Obtained
Medicine	5 kg	50	Full	50
Food	10 kg	30	Full	30
Water	20 kg	20	2 kg out of 20 kg	2
Total	**17 kg**			**82**

Therefore:

Maximum Total Utility = 50 + 30 + 2

Maximum Total Utility = 82

Total Weight Transported = 17 kg

Hence, the boat's complete capacity is utilized and the maximum utility obtained using the fractional greedy approach is **82**.

---

11. Implementation

The source code and assignment files are uploaded to the GitHub repository.

```
Github Link : https://github.com/Ashutoshmore24/DAA_Submission.git
```

12. Complexity Analysis

The time complexity of the Fractional Knapsack algorithm consists of three major operations.

1. Ratio Calculation

The utility-to-weight ratio is calculated for every item.

Time Complexity: **O(n)**

where n is the number of items.

2. Sorting

The items are sorted according to their utility-to-weight ratio.

Time Complexity: **O(n log n)**

3. Selection

The sorted items are traversed once to fill the boat.

Time Complexity: **O(n)**

Overall Time Complexity

Therefore: **O(n) + O(n log n) + O(n)**

Hence, the overall time complexity is: **O(n log n)**

Space Complexity

The algorithm requires storage for the items and their calculated ratios.

Space Complexity: **O(n)**

14. Conclusion

The Fractional Knapsack Algorithm is an efficient Greedy algorithm for maximizing the utility of resources under a limited weight capacity. In the emergency relief supply distribution problem, it helps prioritize critical supplies based on their utility per kilogram.

The algorithm has an overall time complexity of O(n log n), mainly because of sorting. Therefore, the Fractional Knapsack approach is fast, practical, and suitable for resource allocation problems where items can be divided and quick decisions are required, such as emergency relief and disaster management.
